

08 · KI-Beschleuniger & Edge AI

Mikrocomputertechnik 3 · Von der Memory-Wall neuronaler Netze zu Systolic Arrays und TensorFlow Lite

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 73

Willkommen zu Block 8!

- Willkommen zur achten Vorlesung in Mikrocomputertechnik 3
- Block 7: OoO und Branch Prediction maximieren die General-Purpose-CPU
- Heute: Workload Künstliche Intelligenz — und warum Spezialhardware (NPUs) hilft
- Ziel: Memory Wall bei neuronalen Netzen verstehen und den Weg zu DSAs skizzieren



Notizen

General-Purpose-Kerne sind stark bei chaotischem Kontrollfluss. KI: oft Milliarden simpler MACs — andere Hardware-Prioritäten.

Anknüpfung: Compiler-Labor

Kurz-Recap aus Block 7 (Vertiefung im Übungsblatt):

- Godbolt zeigt die Übersetzung, kein OoO-Timing
- -O3 garantiert weder Unrolling noch kürzere Laufzeit
- Scheduling und Unrolling sind mögliche, keine sicheren Folgen
- Eine MAC-Kette bleibt RAW. Mehrere Akkumulatoren können unabhängig sein

```
/* Konzept: Unrolling von for (i = 0; i < 4; i++) a[i] = 0; */  
a[0] = 0;  
a[1] = 0;  
a[2] = 0;  
a[3] = 0;
```

Notizen

Neuronale Netze: lange, vorhersagbare Schleifen — ideal für DLP, teuer für OoO-Overhead.

Grenze der General-Purpose-CPU

- Mehr Issue-Breite verteuert Scheduler, Ports und Bypass. Quadratisch gilt nicht pauschal für ROB, Renaming und Decode zusammen
- OoO kann Loads überlappen. Es hebt eine einzelne Akkumulator-kette nicht auf
- DLP ergänzt ILP. SIMD ist eine Umsetzung, keine Definition von DLP

Notizen

Schweizer Taschenmesser vs. Motorsäge: Excel und OS brauchen Flexibilität; Milliarden MACs brauchen Durchsatz.

Neuer Workload: Edge AI

- Edge AI umfasst Endgerät und Gateway. Nicht jede lokale Inferenz braucht eine NPU
- Beispiele: Schrittzählung, Wake-Word, Vibrationsdiagnose an Maschinen
- Dauerbetrieb auf knappen Ressourcen (Energie, SRAM, Flash)

Notizen

KI verlässt Serverfarmen — trifft auf Knopfzelle und Milliwatt-Budgets.

Cloud AI vs. Edge AI

Lokale und Cloud-Inferenz. Keine Aussage, Funk sei immer teurer.



Anforderungen: Latenz, Energie, Datenschutz, Verfügbareit. Jede kann ausfallen.

Edge umfasst auch Gateways. Nicht jede lokale Inferenz braucht eine NPU.

Lokal bleiben heisst nicht automatisch datenschutzkonform oder sicher.

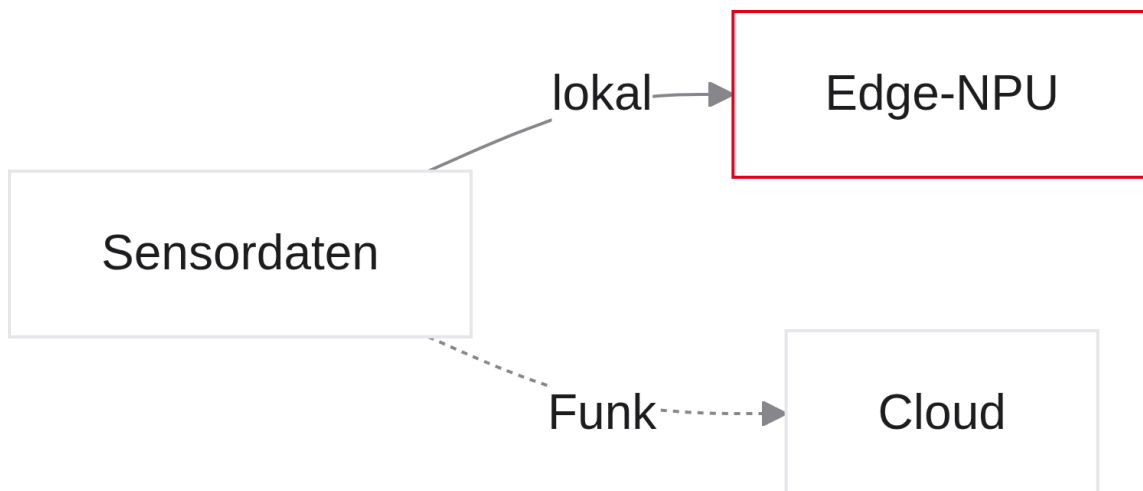
OoO kann Load-Latenzen und mehrere Akkumulatoren nutzen. Eine einzelne Kette bleibt RAW.

-O3 garantiert weder Unrolling noch kürzere Laufzeit.

Abbildung 1: Lokal und Cloud sind Optionen. Funk ist nicht immer die teurere Seite.

Warum nicht alles in die Cloud?

1. Latenz — sicherheitskritische Reaktionen dulden keine Funkwartezeit
2. Privacy — Audio/Video sollen oft lokal bleiben
3. Energie und Verfügbarkeit sind Anforderungen. Funk ist nicht immer teurer als lokale Rechnung



Notizen

Lokal bleiben garantiert weder Datenschutz noch sicheren Betrieb. Cloud und Edge sind Entwurfsoptionen.

Agenda

1. KI-Herausforderung und Memory Wall
2. DSPs und SIMD
3. RISC-V Vector Extension (RVV)
4. NPU und Systolic Arrays
5. Edge-AI-Software / TensorFlow Lite

Notizen

Von Memory Wall über RVV und Systolic Arrays bis TFLM am Edge.

Lernziele Block 8

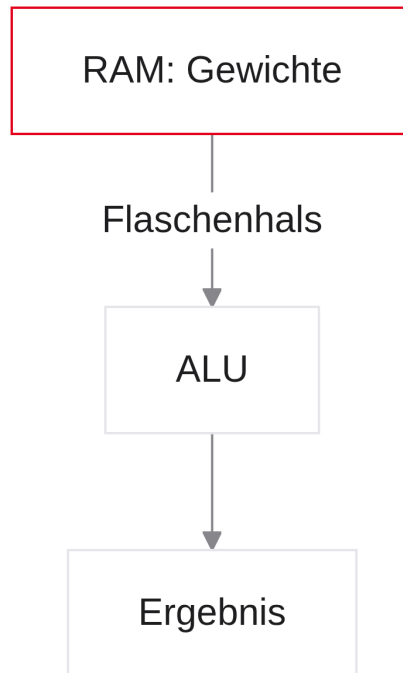
- Memory Wall bei Matrix-/MAC-Lasten erklären
- MAC-Operation und ihre Bedeutung benennen
- ILP vs. DLP unterscheiden
- Systolic Arrays einordnen
- TensorFlow Lite Micro Workflow skizzieren

Notizen

Ziel: verstehen, warum NPUs/TPUs anders gebaut sind als OoO-Kerne.

Die Leitfrage des Tages

- Kleines Netz: z. B. 10^6 Gewichte — Inferenz verrechnet sie mit Aktivierungen
- SRAM, Cache-Hit, Flash und DRAM haben verschiedene Kosten. Nicht jeder Zugriff ist ein DRAM-Miss
- Wie rechnet man Millionen MACs/s auf knapper Energie, ohne am Speicher zu ersticken?



Notizen

Zentrale Spannung: Datenvolumen vs. Speicherlatenz/Energie auf dem Edge-SoC.

Aufgabe — Sechs MACs

Warum sind sechs Gewichte hier sechs MACs, bei einer Faltung aber nicht? Was ist y_0 ?

Notizen

Jedes Gewicht wird einmal mit einem Input multipliziert. y_0 ist 14. Eine Faltung verwendet Gewichte mehrfach, deshalb übersteigt die MAC-Zahl die Parameterzahl. Siehe die Dense-Abbildung.

Neuronale Netze auf Hardware-Ebene

- Abseits des Hypes: große Zahlenarrays
- Inputs / Activations: Sensordaten (z. B. 28×28 Pixel)
- Weights: trainierte Konstanten (oft Flash)
- Neuron: gewichtete Summe der Eingänge (+ Aktivierung, hier ausgeblendet)

Notizen

Für Architekten: Listen von Zahlen und MAC-Schleifen — kein „Gehirn“-Mythos nötig.

Mathematik: Matrix / Dot Product

Dense $y = Wx + b$. W ist 2×3 , x hat 3, b und y haben 2. Integer-Analogie, kein Float-mul.

$W = \begin{bmatrix} 1 & 2 & 3 \\ 0 & -1 & 4 \end{bmatrix}$, $x = \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix}$, $b = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$

$y_0 = 1 \cdot 2 + 2 \cdot 1 + 3 \cdot 3 + 1 = 14$

$y_1 = 0 \cdot 2 + (-1) \cdot 1 + 4 \cdot 3 + (-2) = 9$

Parameter $6+2 = 8$. MACs nur die Produkte: 6. Nicht allgemein Parameter = MACs.

1 MAC = 2 arithmetische Ops in dieser TOPS-Konvention. Bias-Add ist extra.

Parameter 8	MACs 6	Arithmetik 14	Int8-Bytes Daten 11
----------------	-----------	------------------	------------------------

$14 = 6 \cdot 2 + 2$ Bias-Additionen. Schleife und Pointer zählen hier nicht mit.

RV32IM mul/add sind Ganzzahl. Eine Float32-MAC wäre eine andere ISA.

FMA rundet einmal. Getrennte Mul und Add runden zweimal.

Convolution: ein Gewicht wird mehrfach benutzt. Dann sind MACs grösser als die Parameterzahl.

10^6 Gewichte sind nur unter der Annahme ein Gewicht pro MAC auch 10^6 MACs.

Abbildung 2: $y_0 = 14$ und $y_1 = 9$. Sechs MACs, acht Parameter, 14 arithmetische Operationen.

```
float sum = 0.0f;
for (int i = 0; i < N; i++) {
    sum += input[i] * weights[i];
}
```

- Kein Kontrollfluss-Chaos: vorhersagbarer Datenstrom
- Pro Iteration: zwei Loads + Mul + Add

Notizen

Simpler Code — brutal für den Speicherbus.

MAC-Operation

- Multiply-Accumulate: $a \leftarrow a + (w \times x)$
- w : Gewicht, x : Activation, a : Akkumulator
- KI-Chips: oft in TOPS (viele MAC-Äquivalente / s) spezifiziert — nicht nur in MHz

Notizen

MAC ist das Atom der Inferenz-Hardware.

Anatomie einer MAC auf RISC-V

Vereinfachte Befehlsfolge:

```
# Konzept - RV32IM (mul); Adressen schematisch
lw  t0, 0(s1)      # x
lw  t1, 0(s2)      # w
mul t2, t0, t1
add t3, t3, t2      # a += w*x
```

- mul/add sind hier Ganzzahl, die C-Schleife darüber ist Float. Das ist eine Analogie, keine Float-ISA
- Zwei Loads und zwei Ops gelten nur ohne Wiederverwendung und ohne FMA

Notizen

Ohne Wiederverwendung der Daten verhungert die ALU am Bus.

Arithmetic Intensity: Zahlenbeispiel

Lehr-Roofline, keine Chipmessung. Grenze: DRAM zu On-Chip. Peak 8 Ops/Takt, Band 8 Byte/Takt.
Knick bei Peak/Bandbreite = 1 Op/Byte.



Naiv: $2 \text{ Ops} / 8 \text{ Byte} = 0,25$. Erreichbar $\min(8, 8 \cdot 0,25) = 2 \text{ Ops/Takt}$.

2x2 ohne Wiederverwendung: 64 Byte, untere Schranke 8 Takte bei 8 Byte/Takt.

Mit lokalem Tile: 32 Byte, Schranke 4 Takte. Rechnung $\min(\text{Peak}, \text{Band} \cdot \text{AI})$.

Nicht jeder Zugriff ist ein DRAM-Miss. SRAM, Cache-Hit und Prefetch sind andere Faelle.

Abbildung 3: Knick bei 1 Op/Byte. Naiv 0,25, mit Wiederverwendung 0,5. Lehrwerte, keine Messung.

- $AI = \text{Ops}/\text{Bytes}$
- Grenze dieses Beispiels: DRAM zum On-Chip-Speicher. Output und Bias zählen hier noch nicht
- 2 Ops und 8 Byte ergeben 0,25 Ops/Byte. Das ist nicht allgemein „unter 1 = memory-bound“
- Erreichbar ist $\min(P_{peak}, \text{Bandbreite} \times AI)$. Der Knick liegt bei $P_{peak}/\text{Bandbreite}$

Notizen

Im Lehrmodell mit 8 Ops/Takt und 8 Byte/Takt liegt der Knick bei 1 Op/Byte. Andere Maschinen haben einen anderen Knick.

Datenproblem: Gewichte

Dieselbe 2x2-Matmul. Lokaler Speicher fasst A und B, 8 Elemente.

Naiv, kein Halten
8 Produkte, jedes A und B neu
16 Float32-Loads = 64 Byte

Ein Tile, alles lokal
A und B einmal laden: 8 Floats
32 Byte extern, Produkte aus dem Tile

Gewichte, Aktivierungen und Teilsummen getrennt zaehlen.

Zwischenaktivierungen koennen mehr Speicher brauchen als die Gewichte.

Systolische Weitergabe hilft lokal. Den externen Verkehr beseitigt sie nicht.

Annahme: Element bleibt im Tile, kein Write-Allocate, Output-Bytes zunaechst getrennt.

CPU und GPU koennen dasselbe Blocking. Das Array ist eine Hardwareform davon.

Abbildung 4: Dieselbe 2x2-Rechnung: 64 Byte naiv, 32 Byte aus einem Tile.

- Modelle wie ResNet-50: Millionen Parameter — Megabyte-Bereich
- L1 oft nur Dutzende KiB → Matrizen passen nicht
- Ein Streaming-Datensatz erzeugt nicht bei jedem Skalarzugriff einen Capacity Miss. Cachezeilen und Prefetch bündeln Zugriffe

Notizen

Block-2-Caches helfen bei Lokalität — bei einmaligem Durchlauf riesiger Matrizen kaum.

Memory Wall bei KI

Phase	Typische Dauer (schematisch)
Load x, w (DRAM)	viele Takte
MUL + ADD	wenige Takte
nächster Load	wieder viele Takte

- ALU oft idle; Energie geht in Wartezeit und Datentransport

Notizen

Diagramm des Grauens: Mathematik ist billig, Daten holen ist teuer.

Rechenbeispiel: 10^6 MACs

- 10^6 Gewichte sind nur dann 10^6 MACs, wenn jedes Gewicht genau einmal multipliziert wird
- Eine Faltung nutzt Gewichte mehrfach. Sequenz, Bild und Batch ändern den Aufwand
- Einzelne Miss-Latenzen darf man nicht zu einer Summe addieren, wenn Zugriffe überlappen

Notizen

Selbst mit Prefetch bleibt der Von-Neumann-Pfad der Engpass.

Warum OoO hier wenig hilft

- OoO/Predictor: stark bei unregelmäßigem Kontrollfluss
- MAC-Schleife: linear, hoch vorhersagbar
- Der Predictor ist bei einer festen Schleife nicht wertlos, aber selten der Engpass
- Unabhängige Akkumulatoren kann OoO ausführen. Die Speichergrenze bleibt, wenn die Arithmetic Intensity niedrig ist

Notizen

Ironie: Cleverness aus Block 7 ist für diesen Workload oft Verschwendung.

Fazit: CPU vs. KI-Last

- ALU-Auslastung bei naiver Matrixrechnung oft gering
- Viel Zeit/Energie in Transport und Instruktionsverwaltung
- Edge-KI braucht andere Datenpfade — Richtung Domain-Specific Architecture

Notizen

Verhältnis Rechenwerk zu Datenbeschaffung muss radikal anders werden.

Lösung: Data-Level Parallelism (DLP)

- Weg von „viele verschiedene Instruktionen“ (ILP)
- Hin zu: eine Instruktion auf vielen Daten gleichzeitig (DLP)
- Viele unabhängige MACs → viele Units, ein Kommando

Notizen

100 unabhängige Multiplikationen: nicht nacheinander — parallel mit einem Issue.

Domain-Specific Architecture (DSA)

- Extreme DLP-Optimierung → Verlust der Allzweck-Fähigkeit
- DSA: Silizium für eine Domäne (z. B. Matrizen) — oft deutlich effizienter
- Evolutionsschritte: SIMD → DSP-Tricks → Vektoren → NPUs

Notizen

Nach dem Ende leichter CPU-Gains: Spezialchips als Leistungspfad.

Fragen? — MAC und Memory Wall

- Was leistet die MAC-Operation im Netz?
- Warum kehrt die Memory Wall bei KI zurück?
- ILP vs. DLP?

Notizen

Als Nächstes: SIMD und DSPs — historische und praktische DLP-Stufen.

Was bedeutet SIMD?

- SIMD: Single Instruction, Multiple Data
- Paradebeispiel für DLP
- Ein Fetch/Decode → intern mehrere parallele Operationen

Notizen

Breite Execution hinter einem Befehl — weniger Instruktions-Overhead.

SIMD-Prinzip

Packed Add ist kein Int8-Dot-Product. Lanes haben getrennte Uebertraege.

Lane				
	01 + 02 Ergebnis 03	10 + 20 Ergebnis 30	7F + 01 Ergebnis 80/sat	FF + 01 Ergebnis 00/sat

Int8-MAC: Produkt, dann breiter Akkumulator. Signed/Unsigned und Sättigung festlegen.

Beispiel: $20 * 20$ passt nicht in int8. Akkumulator hier int32.

Ein neuer MAC je Takt ist ein Initiationsintervall, nicht die Ergebnis-Latenz.

Zwei Speicherbusse helfen nur ohne Bankkonflikt.

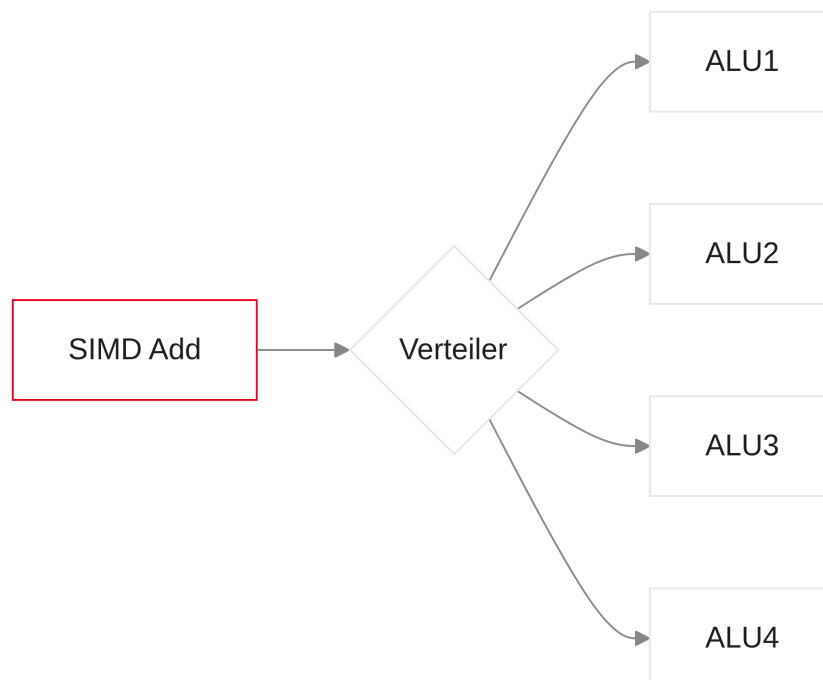
Zero-Overhead-Loop entfernt den Schleifenzweig, nicht jeden Daten-Stall.

SIMD, RVV und NPU sind kombinierbar, keine Pflicht-Reihenfolge.

Die letzten beiden Lanes zeigen Wrap ohne Sättigung beziehungsweise den Sättigungsfall als Alternative.

Abbildung 5: Vier 8-Bit-Lanes addieren getrennt. Ein Int8-Produkt braucht einen breiteren Akkumulator.

- Skalar (SISD): `add` → eine Addition
- SIMD: ein Befehl → mehrere parallele Adds auf Daten-Lanes



Notizen

Fetch/Decode einmal — Execution-Width multipliziert.

Packed SIMD

- Problem: schmale ISA-Register, breite Parallelität?
- Lösung: Register in Lanes zerlegen (z. B. 32 Bit $\rightarrow 4 \times 8$ Bit)
- SIMD-Op: Lane für Lane parallel, ohne Carry zwischen Lanes

Notizen

90er-Hack: vorhandene Datenwege in Scheiben für Audio/Sensorwerte nutzen.

Digital Signal Processors (DSPs)

- Vor dem KI-Boom: DSPs für Audio, Modem, Filter
- Algorithmen (FIR etc.): wie KI stark Array-/Skalarprodukt-lastig
- Vorläufer vieler NPU-Ideen

Notizen

Großväter der NPUs — Datenstrom statt allgemeiner Kontrollfluss.

DSP: dedizierte MAC-Einheit

- Mul und Add oft fest verdrahtet als eine MAC
- $a + (x \times w)$ in wenigen Takten / oft einem Zyklus (je nach Design)
- Weniger Instruktionsfolge als auf skalarer RISC-ALU

Notizen

Werte rein — Summe raus: Hardware für den Hot Path.

Super-Harvard in DSPs

- Mehrere parallele Speicherpfade gegen die Memory Wall
- Typisch: Code-Bus + getrennte Datenbusse für x und w
- Loads und MAC können im selben Takt überlappen

Notizen

Zwei Zahlen brauchen zwei Wege — Architektur folgt dem Workload.

Zero-Overhead Loops

- Hardware-Schleifenzähler: Start, Ende, Count
- Kein `beq` pro Iteration — kein Predictor-Overhead
- PC-Wrap in Hardware bis Count abgelaufen

Notizen

Schleifen ohne Software-Branch-Penalty — reine Rechenpower.

Grenze: Packed SIMD skaliert schlecht

- Feste Vektorbreite in der ISA (z. B. 128 / 256 / 512 Bit)
- Alter Code nutzt neue breitere Units nicht automatisch
- Oft Neukompilieren / Intrinsics für jede Generation

Notizen

Starrheit: Software an Bitbreite gekoppelt.

Instruction-Fetch-Overhead bleibt

- Auch breites SIMD: weiterhin viele Instruktionen für große Matrizen
- Decode und I-Cache kosten Energie
- Wunsch: „rechne lange auf diesen Daten“ mit weniger Instruction Stream

Notizen

SIMD lindert — heilt den Instruktions-Overhead nicht vollständig.

Übergang zu Vektoren

- Weg von starrem Packed SIMD → flexible Vektorlängen
- RISC-V Vector Extension (RVV): Ansatz für skalierbare Vektor-ISAs
- Code-Idee: dieselbe Software auf schmalen und breiten Implementierungen

Notizen

RVV und danach Systolic Arrays / NPUs.

Kurzfazit: DSP und SIMD

- DLP: ein Befehl, viele Daten
- Packed SIMD: breite Register, starre Breiten
- DSPs: MAC, Multi-Bus, Zero-Overhead Loops
- Für moderne KI-Arrays oft noch zu starr / zu instruction-lastig → Vektoren & NPUs

Notizen

Pioniere der Datenparallelität — Grundlage für heutige Beschleuniger.

Fragen? — SIMD und DSP

- Wie senkt SIMD den Fetch/Decode-Overhead?
- Warum ist eine Hardware-MAC für Signalverarbeitung wertvoll?
- Warum muss Packed-SIMD-Code oft neu angepasst werden?

Notizen

Als Nächstes: RISC-V Vectors, Systolic Arrays, TensorFlow Lite Micro.

Echte Vektor-Architektur

- Verarbeitet lineare Arrays (Vektoren), nicht nur Skalare
- Anders als Packed SIMD: Vektorlänge flexibel, nicht in feste Registerhäppchen geschnitten
- Befehlssatz abstrahiert: „Addiere Vektor A und B“ — unabhängig von der Implementierungsweite

Notizen

Seit Cray-Supercomputern bekannt; RISC-V bringt das Konzept zurück in die MCU-Welt.

Vektor-Register-File

- Neben $x0-x31$: Vektorregister $v0-v31$
- Jedes v -Register hält viele Elemente gleichzeitig
- Kapazität hängt von der Implementierung ab ($VLEN = \text{Vector Length}$)

Notizen

32 „Güterzüge“ neben den Skalar-Kisten — ein Vektorbefehl arbeitet auf dem ganzen Zug.

RISC-V Vector Extension (RVV)

- Offizielle RISC-V-Erweiterung für Vektormathematik (u. a. ML, Krypto)
- Neue Instruktionen, z. B. `vadd.vv`
- Software kennt die physische Breite oft nicht (Watch vs. Server)
- Frage: ein Binary für unbekannte VLEN?

Notizen

Anders als AVX: Code nicht an feste Bitbreite ketten.

`vsetvli` / hardware-agnostisch

- Software: „Array mit N Elementen“
- Bei $LMUL = 1$ ist $VLMAX = VLEN/SEW$. Allgemein $VLMAX = LMUL \times VLEN/SEW$
- `vsetvli` liefert nicht immer $\min(AVL, VLMAX)$
- Schleife läuft in Hardware-Schritten durch das Array

RVV 1.0, LMUL=1, e32, ta, ma. Nicht in diesem Repo assembliert
loop:

```
vsetvli t0, a0, e32, m1, ta, ma
vle32.v v1, (a1)
vle32.v v2, (a2)
vadd.vv v3, v1, v2
vse32.v v3, (a3)
slli t1, t0, 2
add a1, a1, t1
add a2, a2, t1
add a3, a3, t1
sub a0, a0, t0
bnez a0, loop
```

Notizen

Laufzeit-Abfrage der Kapazität — Schrittweite passt sich dem Silizium an.

Ein Binary, viele VLEN

LMUL=1, e32. VLMAX = VLEN/SEW. VLEN ist nicht die Lane-Zahl.

VLEN 128 Bit, VLMAX 4
1000 Elemente: 250 mal VL=4, Rest 0

VLEN 1024 Bit, VLMAX 32
31 mal VL=32 und einmal VL=8

$31 \cdot 32 + 8 = 1000$. Zusammen 32 Schleifen, nicht 31.

vsetvli liefert nicht blind $\min(\text{AVL}, \text{VLMAX})$, wenn $\text{VLMAX} \leq \text{AVL}$; $2 \cdot \text{VLMAX}$.

Hier ist der Rest 8 $\leq \text{VLMAX}$, also VL=8 erlaubt und in diesem Modell gewählt.

Eine physische Lane-Gruppe kann VLMAX Elemente ueber mehrere Takte rechnen.

Ein Binary braucht passende ISA, Erweiterung, ABI und einen RVV-Kern.

Pointer += VL * 4 Byte. Zaehler -= VL. Ende bei AVL 0.

Policy im Beispiel: ta, ma, m1. Spezifikation RVV 1.0.

Keine Toolchain in diesem Repository hat die Schleife assembliert.

Abbildung 6: 128 Bit: 250 Schritte zu VL 4. 1024 Bit: 31 Schritte zu VL 32 plus VL 8.

Beispiel: 1000 Elemente, `vadd.vv` (schematisch)

Plattform	Breite (schematisch)	VL (e32)	Schleifenläufe
Edge	schmal (z. B. 128 Bit)	4	250
Server	1024 Bit, LMUL 1	32, zuletzt 8	32

Gleiche RVV-Schleifenidee skaliert mit VLEN — ohne feste Packed-SIMD-Breite im Quelltext.

Notizen

1000/32 ist nicht teilbar: 31 mal VL 32 und einmal VL 8. Ein Binary braucht ISA, ABI und einen Kern mit RVV.

Energie: weniger Instruction Fetch

- Ein Vektorbefehl → viele Elemente parallel
- Skalar: dutzende Loads/Adds/Branches aus dem I-Cache
- Weniger Fetch kann Energie sparen und garantiert keinen geringeren Gesamtverbrauch

Notizen

Speicher- und Registerverkehr bleiben in der Energiebilanz. Chaining ist Mikroarchitektur, nicht die ISA.

Vector Load/Store

- Breite Speicherpfade zum D-Cache / SRAM
- `vle32.v`: zusammenhängender Block → Vektorregister
- Ähnlich Burst-Idee: DRAM-/Bus-Lokalität besser ausnutzen

Notizen

Speicher muss mitwachsen — ein Load füllt den Güterzug.

Chaining

Lehrlatenzen: Mul 2 Takte, Add 1. Vier Elemente, eine Lane. Keine RVV-Hardwarezeit.

Getrennt: Mul fertig ab Takt 2, Add erst danach. Alles fertig Takt 6.

Chaining: Element 0 geht nach Mul-Latenz in den Add. Folgeelemente ueberlappen.

Erstes Ergebnis Takt 3. Letztes Takt 6. Bandbreite kann vorher begrenzen.



Annahme: Initiationsintervall 1, eine Lane, Forwarding nur im Chaining-Fall.

RVV garantiert Chaining nicht. FMA ist zusätzlich eine andere Rundung.

Weniger Fetch spart nicht automatisch Gesamtenergie. Speicher bleibt im Budget.

Registerbreite allein erklärt grosse Modelle nicht.

Abbildung 7: Mul-Latenz 2, Add-Latenz 1, eine Lane. Chaining ist keine RVV-Garantie.

- MAC: Mul dann Add — Zwischenergebnis sonst zurück ins Registerfile
- Chaining ist eine Option des Kerns. Eine FMA ist eine andere Instruktion mit einer Rundung
- Weniger Registerverkehr, höherer Durchsatz

Notizen

Pipelining im Rechenwerk: Zwischenergebnisse fließen weiter, ohne zu „parken“.

RVV und TinyML

- TinyML: Inferenz auf sehr knappen Energie-/SRAM-Budgets
- RVV-fähige Kerne: flexibel (noch volle CPU) + Vektor-Durchsatz
- Typisch: Keyword Spotting, kleine Sensor-Modelle am Edge

Notizen

Mittelweg: C/OS auf der CPU, heiße Mathe auf Vektorunits.

Grenzen von Vektoren

- Registerfile bleibt: lesen/schreiben kostet Energie und limitiert Skalierung
- Sehr große Modelle: Pfad ALU → Register wird Engpass
- Nächste Stufe: Daten fließen ohne ständiges Zurückschreiben — Systolic Arrays

Notizen

Wenn Matrizen absurd wachsen, reicht „Vektor + Register“ nicht mehr.

Kurzfazit RVV

- Ein Befehl, lange Arrays
- VLEN/VL abstrahiert Hardwarebreite
- Binary skaliert von Edge bis Server
- Weniger Front-End-Last (DLP)
- Loads und Chaining senken interne Wartezeiten

Notizen

Elegante DLP-Integration in eine Allzweck-ISA.

Aufgabe — Rest 8

VLMAX ist 32, AVL ist 1000. Wie viele Iterationen und welches letzte VL entstehen, wenn der Rest klein genug für ein VL bleibt?

Notizen

31 mal 32 und einmal 8. Zusammen 32 Iterationen und 1000 Elemente. VL ist nicht in jedem AVL-Bereich $\min(\text{AVL}, \text{VLMAX})$. Siehe die RVV-Abbildung.

Was ist eine NPU?

- Neural Processing Unit: Beschleuniger für Inferenz (manchmal Training)
- Namen: TPU (Google), Neural Engine (Apple), NVDLA, ...
- Kein Allzweck-if/else-OS — starrer Matrix-Datenfluss
- CPU füttert Adressen/Puffer (ähnlich DMA-Aufträgen)

Notizen

Radikale DSA: nur Matrizen — dafür extrem effizient.

NPU vs. GPU vs. CPU

	Optimierung	Stärke	Schwäche
CPU	Latenz / ILP	Kontrollfluss, Flexibilität	Array-Mathe ineffizient
GPU	Durchsatz / SIMT	viele parallele Threads	Energie, Kosten
NPU	Datenfluss / DLP	sehr viele MACs/Takt, effizient	starr

Edge: GPUs oft zu hungrig — NPUs für den Rand des Netzes.

Notizen

GPU kann Matrizen — NPU ist darauf zugeschnitten (oft mW-Regime).

Folie

49 / 73

Flaschenhals: Register / Verdrahtung

- Wunsch: zehntausende MACs/Takt
- Jede MAC braucht Operanden — unmöglich, unendlich viele Ports zum SRAM
- Routing und Multiplexer begrenzen die Breite
- Lösung: Daten lokal zwischen Zellen weiterreichen

Notizen

65k Multiplizierer druckbar — 65k dicke Kabel zum Cache nicht.

Folie

50 / 73

Systolic Arrays

- 2D-Gitter aus MAC-Zellen
- Nur der Rand spricht mit dem Speicher
- Ergebnisse/Operanden wandern zu Nachbarn — ohne Systembus pro Schritt

Notizen

Eimerkette: niemand rennt zum Brunnen, jeder reicht weiter.

Folie

51 / 73

„Systolisch“ — Namensbild

- Systole: rhythmische Pumpwelle
- Takt pumpt Werte wellenartig durch das Gitter
- Pro Zyklus: Daten eine Zelle weiter (rechts/unten je nach Dataflow)

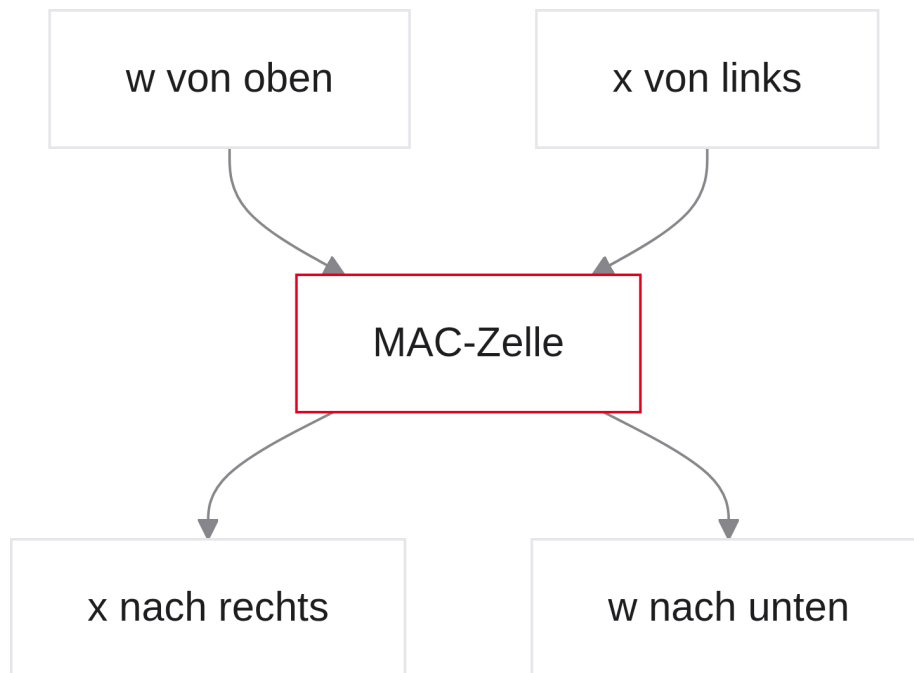
Notizen

Oszillator = Herzschlag für Nullen und Einsen im Array.

Eine systolische Zelle

Pro Takt typisch:

1. Empfängt x (links) und w (oben)
2. MAC auf interne Teilsumme
3. Reicht x nach rechts und w nach unten weiter



Notizen

Einfach, aber entscheidend: Originalwerte durchreichen für Wiederverwendung.

Weniger RAM-Zugriffe

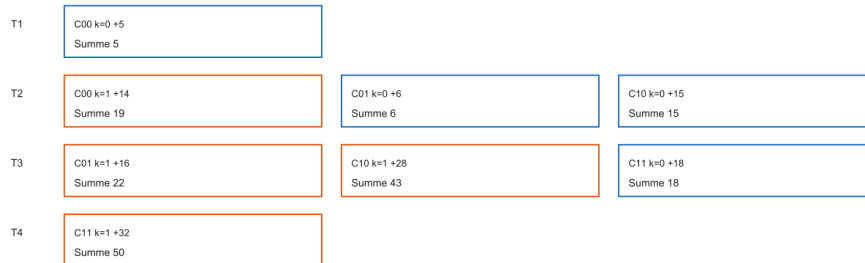
- Ein Wert wird einmal eingespeist und wandert durch viele Zellen
- Viele MACs ohne erneuten Cache-/DRAM-Load
- Arithmetic Intensity steigt — Memory Wall entschärft

Notizen

Load ist teuer; lokale Wiederverwendung ist der Triumph.

Datenfluss füllt das Array

Output-Stationary: A nach rechts, B nach unten, C bleibt. Takt = ein gültiger MAC.
C00=19, C01=22, C10=43, C11=50. Fertig in den Takten 2, 3, 3 und 4.



Nicht: ab Takt 3 ein fertiger Output je Takt. C11 endet erst in Takt 4.

Staffelung: gleiche k treffen sich, weil Zeile und Spalte verzögert eingespeist werden.

Fill Takt 1, Compute bis Takt 4, Readout der lokalen C danach. Drain ist kein Automatismus.

Leere Zellen in einem Takt sind ungültige Begegnungen, keine extra MACs.

Abbildung 8: C00 fertig in Takt 2, C01 und C10 in Takt 3, C11 in Takt 4.

2×2 -Idee (schematisch):

- Takt 1: erste Paare treffen sich, Summen starten
- Takt 2: Wellen wandern, neue Werte am Rand
- Im 2×2 -Output-Stationary-Beispiel enden die vier Summen in den Takten 2, 3, 3 und 4
- Das ist kein Ergebnis je Takt ab Takt 3. Readout der lokalen Summen kommt danach

Notizen

Wie Instruction-Pipeline: erst füllen, dann kontinuierlicher Ausstoß.

Weight-Stationary

Zwei Dataflows. Der Rand-Speicherzugriff gilt fuer dieses vereinfachte Array.

Output-Stationary

C bleibt. A horizontal, B vertikal.
Teilsomme lokal. Readout am Ende.
Gewichte fließen, sie stehen nicht fest.

Weight-Stationary

Gewicht steht. Aktivierung fließt.
Teilsomme kann weiterwandern.
Weights brauchen trotzdem Load und Wechsel.

Nicht jede NPU ist systolisch und nicht jede rechnet nur Matrizen.

Vergleich CPU/GPU/NPU braucht Datentyp, Batch, Auslastung und Transfer.

TPU v1: $256 \times 256 \times 2 \times 700e6 = 91,75e12$ Ops/s, oft 92 TOPS genannt. Datacenter, kein mW-Edge.

Peak ist nicht die End-to-End-Inferenzleistung.

Abbildung 9: Output-Stationary hält C. Weight-Stationary hält das Gewicht und muss es trotzdem laden.

- Gewichte während einer Inferenz oft konstant
- Variante: Weights in Zellen „parken“, nur Activations durchschieben
- Weniger Bewegungen der großen Konstanten-Matrizen

Notizen

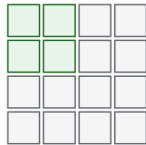
Konstanten einmal setzen — dynamische Inputs fluten das Gitter.

Beispiel: Google TPU v1

Peak-Rechnung und illustrative Auslastung. Keine gemessenen TOPS/W.

$65536 \text{ MACs} \cdot 700 \text{ MHz} = 45,8752 \text{ TMAC/s}$. Mal 2 Ops = 91,7504 TOPS.

2x2-Rechnung auf einem gedachten 4x4-Array: 4 von 16 Zellen aktiv = 25 Prozent.



Grün = aktive Zelle des 2x2-Beispiels.

Prüfliste: Präzision, Ops je MAC, Sparse oder Dense, Batch, Messgrenze, End-to-End-Latenz.

Nullen in einer Dense-Matrix sparen keine Zeit, solange der Kernel sie rechnet.

Tile, Fill, Drain und nicht beschleunigte Operatoren senken die Auslastung.

Abbildung 10: 91,75 TOPS Peak bei zwei Ops je MAC. Ein 2x2-Tile nutzt 4 von 16 gedachten Zellen.

- $256 \times 256 = 65\,536$ Int8-MACs. Bei 700 MHz sind das 45,8752 TMAC/s
- Mit zwei Ops je MAC: 91,7504 TOPS, in der Originalarbeit oft 92 TOPS. Das ist Peak, kein End-to-End
- TPU v1 ist ein Datacenter-Baustein, kein Milliwatt-Edge-Teil

Notizen

Weckruf: fast nur Multiplikator-Gitter — Domäne statt Alleskönner. Marketing-TOPS von der Zählkonvention abhängig.

NPU am Edge

- SoCs mit kleineren Arrays (z. B. 32×32 / 64×64)
- CPU: OS, Netz; NPU: Bild/Audio-Inferenz (oft per DMA gefüttert)
- Wake-Word & Co.: NPU arbeitet, CPU kann schlafen

Notizen

Technologie geschrumpft — Systolic Arrays in günstigen Edge-Chips.

Grenzen systolischer Arrays

- Starr an Array-/Tile-Größen — Padding bei ungünstigen Layern
- Dense-Matrizen stark; Sparse (viele Nullen) weniger effizient
- Kein General Purpose — kein TCP/IP-Stack auf der NPU

Notizen

Spezialist: falsch dimensionierte Layer → brachliegende Zellen.

Kurzfazit NPU

- Systolic Arrays: lokale Weitergabe statt Von-Neumann-Pendeln
- Wellenartiger Datenfluss, hohe Wiederverwendung
- Weight-Stationary spart Bewegungen
- Hohe TOPS/Watt — nur für Matrix-Domäne

Notizen

Hardware verstanden — als Nächstes Software-Brücke zum MCU.

Aufgabe — Takt 4

Wann ist C11 im 2×2-Output-Stationary-Beispiel fertig? Ist 92 TOPS eine End-to-End-Leistung?

Notizen

C11 wird in Takt 4 zu 50. 91,75 TOPS sind der Peak aus 65536 MACs, 700 MHz und zwei Ops je MAC. Siehe Array- und TOPS-Abbildung.

Edge-AI-Software: das Problem

- Training oft in Python (TensorFlow/PyTorch) auf GPUs — große Float32-Modelle
- MCU: kein Python, knappes SRAM, oft keine FPU
- Wie wird das Cloud-Modell zu Bare-Metal-C?

Notizen

Zwei Welten: unbegrenzte Cloud vs. jedes Kilobyte und jeden Takt zählen.

TensorFlow Lite for Microcontrollers (TFLM)

- Kleine C++-Runtime für MCUs (ohne OS, ohne `malloc`)
- Nimmt kompaktes Modell, interpretiert Layer (Conv2D, Dense, ...)
- Rechnet mit Firmware-Kernen. Eine NPU braucht ein passendes Backend, nicht nur das Silizium

Notizen

Feste Arena im SRAM — kein dynamisches Heap-Wachstum.

Workflow Embedded AI

Workflow. Das Byte-Array ist Modelldaten, kein ausführbarer Maschinencode.



CPU-Kernel ist der Default. Eine NPU braucht Backend, Operator und Toolchain.

AllocateTensors und Invoke haben Rueckgaben. Arena-Groesse wird gemessen, nicht geraten.

Int8-Input folgt scale und zero_point der Trainingsvorverarbeitung.

Ein Logit ist keine kalibrierte Wahrscheinlichkeit. 0,8 ist keine allgemeine Schwelle.

Renode-Funktion ist kein Beweis fuer Memory Wall, CPI oder NPU-Beschleunigung.

Abbildung 11: Das C-Array sind Modelldaten. Invoke läuft über Kernel, nicht als Modell-Maschinencode.

1. Trainieren (Cloud / Python)
2. Konvertieren → `.tflite`
3. Export als C-Byte-Array (`xxd` o. Ä.)
4. Linken in Flash (`.rodata`)
5. Firmware: `Invoke()` / Interpreter

Notizen

Laboridee: fertiges Modell als Hex-Array in die Firmware schweißen.

Quantisierung

$real = scale * (q - zero_point), scale=0,1, zero_point=0$. Rundung zur nächsten Zahl.

1,24 -> q=12 -> zurück 1,2. Fehler 0,04.

20,0 -> Rohwert 200 -> Clamp 127 -> zurück 12,7.

Int8-Gewichte: 4 Byte auf 1 Byte, 75 Prozent weniger reine Gewichtsbytes.

Bias, Metadaten, Aktivierungen, Scratch und Arena bleiben extra.

Flash: Gewichte
nur der konstante Block

SRAM: Input, Output, Aktivierungen, Scratch
Arena ist dieser Puffer, kein zweiter Heap

Int32-Akkumulator fuer geeignete Int8-MACs. Per-Tensor und Per-Channel sind verschiedene Scales.

Integer ist nicht auf jeder CPU schneller. Accuracy gehoert auf Testdaten gemessen.

Pruning aendert die Darstellung. Weniger MACs nur, wenn der Kernel Sparse versteht.

Kein 10-KiB- oder 20-KB-Wert ohne Messung.

Abbildung 12: 1,24 wird 12 und zurück 1,2. 20 wird auf 127 begrenzt. Flash und SRAM bleiben getrennt.

- Float32: 4 Byte/Gewicht, teuer ohne FPU
- Reine Float32-Gewichte werden als Int8 um 75 Prozent kleiner. Bias, Aktivierungen und Arena nicht automatisch
- Integer ist nicht auf jeder CPU schneller. Den Accuracy-Verlust messen, nicht annehmen

Notizen

Netze sind robust gegen Quantisierungsrauschen — oft akzeptabler Trade-off.

Pruning

- Viele Gewichte nahe null nach dem Training
- Pruning braucht eine sparsityfähige Darstellung und einen passenden Kernel
- Flash-Ersparnis und weniger ausgeführte MACs sind zwei verschiedene Folgen

Notizen

Tote Äste entfernen — näher an der Memory Wall vorbei. Oft im Trainingsgraphen, nicht im C-Compiler.

Laborausblick: KI-Schrittzähler

Geplant (Details im Übungsblatt):

- Kleines quantisiertes Modell (< 20 KB)
- Eingabe: 3-Achsen-Beschleunigung (Anknüpfung I2C, Block 5)
- Klassen Stehen, Gehen, Laufen sind eine Aktivitätserkennung, noch kein Schrittzähler
- Simulation: Renode mit virtuellen Sensordaten

Notizen

Pipeline: Sensor → TFLM → Klasse — Busse + Edge AI zusammen.

Modell als C-Array

```
/* Intelligenz als .rodata im Flash */  
const unsigned char step_model_tflite[]  
    __attribute__((aligned(4))) = {  
    0x1c, 0x00, 0x00, 0x00, 0x54, 0x46, 0x4c, 0x33,  
    /* ... weitere Bytes ... */  
};
```

Notizen

Die Bytes sind Modelldaten. Kernel in der Firmware führen Operatoren aus. Flash-Lesen ist möglich, wenn Plattform und Kernel das erlauben.

Tensor Arena

```
const int kTensorArenaSize = 10 * 1024;
uint8_t tensor_arena[kTensorArenaSize];

/* Lehrschnitt, hier nicht übersetzt. Kein TFLM-Tree im Repository
   Konstruktor ohne ErrorReporter, Rückgaben prüfen. */
if (model->version() != TFLITE_SCHEMA_VERSION) { /* Abbruch */ }
tflite::MicroInterpreter interpreter(
    model, resolver, tensor_arena, kTensorArenaSize);
if (interpreter.AllocateTensors() != kTfLiteOk) { /* Arena */ }
```

- Statischer SRAM-Block — keine Heap-Leaks
- Zu kleine Arena → klarer Fehler bei Allokation

Notizen

Sicherheit durch feste Grenzen statt stiller Stack-Korruption.

Inferenz-Zyklus

```
/* Int8-Lehrschnitt. 128 ist kein definiertes 3-Achsen-Fenster. */
if (input->type != kTfLiteInt8) { /* Typfehler */ }
for (int i = 0; i < input->bytes; i++)
    input->data.int8[i] = quantize(sample[i]);
if (interpreter.Invoke() != kTfLiteOk) { /* Invoke-Fehler */ }
int8_t logit = output->data.int8[2];
/* logit ist kein kalibrierter Score und 0,8 keine Schwelle */
```

- Rohdaten → Tensor → Invoke() → Scores

Notizen

Weniger handgeschriebene Filter — schwarze Kiste mit Wahrscheinlichkeiten.

Renode: virtuelle Stimuli

- Wie Block 5: Muster in `.resc` / Sensor-Modell einspeisen
- Firmware liest I2C-Register, füttert TFLM
- Erwartung: Klassifikation in der Konsole

Notizen

Kreis: Emulation, Bus, Modell.

Erwartung: CPU ohne NPU

- Simulation oft RV32IM ohne RVV/NPU → `Invoke()` teuer
- Eine funktionale Simulation belegt weder die Memory Wall noch eine NPU-Beschleunigung
- Motiviert, warum SoCs NPUs neben die CPU setzen

Notizen

Absichtlich „falsche“ Architektur — Flaschenhals im Terminal erleben.

Zusammenfassung Block 8

- KI-Inferenz: Massen an MACs / Matrizen
- CPUs an der Memory Wall bei vorhersehbaren Loads
- DLP → RVV (hardware-agnostisch)
- Systolic Arrays / NPUs: lokale Datenweitergabe
- TFLM + Quantisierung: Modelle in MCU-Flash

Notizen

Edge AI verändert Silizium und Firmware-Workflow.

Ausblick und Abschluss

- Übungsblatt: TFLM / Renode-Schrittklassifikation
- Sensorpipeline aus Block 5 mit Inferenz verbinden

Vielen Dank — viel Erfolg beim Experimentieren.

Notizen

Quellen: RISC-V Vector 1.0, TPU-Paper Jouppi et al. 2017, TFLM und die Int8-Spezifikation von LiteRT.

Im Repository liegen kein TFLM-Tree, kein trainiertes Modell und keine `.repl`.

Wenn das Array plötzlich „Gehen“ erkennt, schließt sich der Kreis von Bus bis NPU-Idee.